

INTEGRANTES

INTEGRANTES - GRUPO 21

- Britez, Lautaro Tomas
- Cabrera, Jorge Sebastian
- Elorrieta, Leandro Agustin
- Fernandez Zalazar, Jeremias
- Garay, Lucia Valentina

Estudio Comparativo del Lenguaje

Estudio Comparativo - Scheme

Scheme

- Scheme es un lenguaje de programación proveniente de la familia Lisp, centrada en el paradigma funcional, pero pudiendo adaptarse a otros estilos, nacida en 1970 en los laboratorios de inteligencia artificial del MIT (Instituto Tecnológico de Massachusetts), caracterizada por su diseño limpio y minimalista: Scheme se conforma por un núcleo muy pequeño, compuesto por pocas reglas teóricas para formar expresiones (dando herramientas precisas para el trabajo deseado), diferenciándola de otros lenguajes con cientos de reglas y estructuras.

- Así como su antecesor Lisp, se clasifica dentro de los lenguajes híbridos, admitiendo secuencias de instrucciones o la asignación de variables, también este mismo es un lenguaje interpretado. Por último, es un lenguaje de programación práctico, eficiente y flexible, pudiendo admitir la mayoría de los paradigmas de programación actuales.

Algunas de las Áreas de Scheme

- Inteligencia Artificial: Aunque a día de hoy abran mejores lenguajes de programación dedicados a la IA, Scheme así como Lisp, son herramientas muy poderosas a la hora de trabajar en sistemas expertos, donde la computadora deduzca respuestas basándose en reglas y relaciones lógicas, pudiendo hacer que el mismo sistema aprenda.

- Ciencia de Datos - Educación: Al ser un lenguaje funcional más minimalista y de pocas reglas, permite la enseñanza de los fundamentos de la programación, así como sus algoritmos; recursivos, abstracción de datos, cálculos lambda, etc.

Empresas que Trabajaron con Scheme

- Como tal Scheme no es un lenguaje muy utilizado en la actualidad, aun así por su simpleza y eficiencia, algunas empresas suelen utilizarlo en áreas de automatización interna o donde se requiera rendimiento por sobre todo.

- Apple: según lo encontrado, se utilizó una implementación interna de Scheme para escribir y configurar reglas del sandbox (entornos de seguridad aislados), para sistemas operativos macOS, y para los iPhone.

- Disney World: Se llegó a utilizar particularmente Chez Scheme como parte del software que operaba el funcionamiento de las atracciones en sus parques, aun así esto fue hace historia.

Diferencias entre Scheme y Lisp

1. Llamadas a las funciones y variables como argumentos:

- Como sabemos en **Common Lisp**, si nosotros queremos pasar una función como argumento a otra función, debemos utilizar `#'` y de la misma manera al recibir esa función como parámetro, dentro de esa misma función debemos de preparar a las variables para recibir las con la siguiente función: **fullcan**, esta se encarga de ejecutar esa función como lo que es y no como una variable.

Esto es debido a que en Lisp, los nombres o símbolos tienen dos lugares en la memoria; Uno para guardar variables y otro para guardar funciones. Al estar separados es posible

tener funciones y variables con los mismo nombres, pero a la hora de pasar funciones como parámetro, se generan esos inconvenientes, el tener que avisar al intérprete que busque en esos lugares respectivos y no genere un error.

- En **Scheme**, por otra parte, como dijimos antes al tener un núcleo tan pequeño y simple donde las reglas y estructuras son más simples y directas. De esta manera, Scheme decidió tener un único espacio para los nombres, en el cual un símbolo solo puede apuntar a una sola cosa a la vez, funciones y variables se tratan como igual, aún sabiendo que esa variable guarda un procedimiento, sigue siendo una variable.

Pero para tener en cuenta como diferenciarlas, en scheme hay una regla y es que **el primer elemento de una x lista siempre se evalúa para ver si es un procedimiento, y el resto sus argumentos.**

Ejemplos:

Common Lisp: Necesita de #' y fullcan para hacer una especie de búsqueda entre los lugares de variables y funciones:

```
1 (defun doble (x) (* x 2))
2
3 ; Usamos #' para sacar realizar la bsqueda en el lugar de funciones
4 (defun aplicar-a-cinco (f)
5   ; Usamos funcall poder ejecutar f como la funcion que llega por argumentos
6   (funcall f 5))
7
8 (aplicar-a-cinco #'doble) ; Devuelve 10
```

Scheme: Trata como igual a funciones y variables, son valores que se guardan en un mismo lugar:

```
1 (define (doble x) (* x 2))
2
3 ; Simplemente ponemos 'f' como primer elemento del paréntesis.
4 (define (aplicar-a-cinco f)
5   (f 5))
6
7 ; No hay #' . Pasamos 'doble' directamente.
8 (aplicar-a-cinco doble) ; Devuelve 10
```

2_ Scheme - Optimización de Llamada de Cola (Tail Call Optimization)

Significa que cuando una función recursiva realiza la llamada recursiva como última operación, el intérprete puede reutilizar el mismo espacio de memoria en lugar de seguir agregando llamadas a la pila.

Por eso, al hacer funciones recursivas en Scheme, se busca que la recursión quede al final de la función, ya que eso ayuda a evitar problemas de desbordamiento de pila cuando se trabaja con listas grandes.

La principal ventaja es que evita problemas de memoria en funciones recursivas grandes. Como desventaja, a veces es necesario reestructurar la función usando acumuladores, lo que puede hacer el código un poco más difícil de entender

Uso de Librerías - Quicklisp

Utilización de Librerías - Local-Time

Que es la librería local-time y cómo funciona:

- local-time, es como sabemos una librería externa para Common Lisp, descargada o implementada mediante un gestor de paquetes llamado Quicklisp, está hecho para manipular y mejorar el horario en general, esto incluye: fechas, horas e intervalos. Este paquete lleva consigo propiedades y objetos más complejos referentes al calendario humano. Es decir, algunas de las que utilizamos a la hora de implementar local-time, fueron:

(format-timestring stream timestamp &key format timezone),

- Función que procesa la información recibida de los parámetros o datos que deseemos mostrar en pantalla.

(unix-to-timestamp unix-epoch)

- Es una función que recibe el formato requerido para nuestra función, los segundos acumulados o formato unix epoch, los ajusta para separarlo en varios objetos.

:format '(:year "-" :month "-" :day " " :hour ":" :min ":" :sec)

- Esta función sería la muestra en pantalla del horario, como dije antes al recibir los segundos acumulados, los ajusta y separa en varios objetos los cuales podemos personalizar, cambiando de lugar o agregando algo después de ellos, como lo muestra ahí.

(local-time:format-timestring nil (local-time:unix-to-timestamp epoch)

:format '(:year "-" :month "-" :day " " :hour ":" :min ":" :sec)

- Y esta sería la función completa que se utilizó en nuestra función.

Porque la librería local-time:

- Como vemos para el requerimiento 3, sistema de auditoría, se requería una forma mas practica de mostrar el horario del cambio de color, por esta cuestion, decidimos implementar de forma obligatoria la libreria local-time, de no ser asi seria mas complicado de leer para las personas comunes, entonces mediante funciones preestablecidas de esta libreria, logramos realizar un horario mucho mas personalizado y de facil lectura.

Otras cuestiones fueron, por lo versátil que era esta librería a la hora de manipular sus funciones, tomando únicamente una función de muestra y otras de conversión para la forma de salida de los objetos que entraban mediante los argumentos o parámetros.

Bitácora - Lautaro Britez

Bitácora:

Britez Lautaro Tomas / Bitácora / Función Estado de Transición:

```
TRANSICION
Entrada:
-color-actual (en-rojo, en-amarillo, en-verde)
-cambiar-a (rojo, amarillo, verde)
Salida:
-lista --> ('en-rojo "cambiar-a-verde")
Comportamiento-Por-Defecto
-transicion-invalida --> ('en-rojo 'accion-por-defecto)
Funcion: Recibir el color actual y cambiarlo al siguiente segun el timer...
```

1er error de lógica:

```
(EN-AMARILLO "cambiar-a-verde")
Break 2 [4]> (transicion 'en-amarillo 'amarillo)

(EN-AMARILLO "cambiar-a-amarillo")
Break 2 [4]> (transicion 'en-rojo 'verde)

(EN-ROJO "cambiar-a-verde")
Break 2 [4]>
```

-El resultado no fue lo esperado, provocando un error de lógica, tal que la salida tendría que ser la lista de función por defecto y no la lista de cambio de color... Código erróneo:

```
(defun transicion (color-actual cambiar-a)
  (cond
    ((equal cambiar-a 'rojo) (list color-actual "cambiar-a-rojo"))
    ((equal cambiar-a 'amarillo) (list color-actual "cambiar-a-amarillo"))
    ((equal cambiar-a 'verde) (list color-actual "cambiar-a-verde"))
    (t (list color-actual "accion-por-defecto"))))
```

-Logre resolverlo realizando una verificación sobre ambos parámetros pasados, lo que funcionó a la perfección...

```
Break 2 [4]> (defun transicion (color-actual cambiar-a)
  (cond
    ((and (equal cambiar-a 'rojo) (equal color-actual 'en-amarillo)) (list color-actual "cambiar-a-rojo"))
    ((and (equal cambiar-a 'amarillo) (equal color-actual 'en-rojo)) (list color-actual "cambiar-a-amarillo"))
    ((and (equal cambiar-a 'verde) (equal color-actual 'en-amarillo)) (list color-actual "cambiar-a-verde"))
    (t (list color-actual "accion-por-defecto"))))

TRANSICION
Break 2 [4]> (transicion 'en-amarillo 'rojo)

(EN-AMARILLO "cambiar-a-rojo")
Break 2 [4]> (transicion 'en-rojo 'amarillo)

(EN-ROJO "cambiar-a-amarillo")
Break 2 [4]> (transicion 'en-amarillo 'verde)

(EN-AMARILLO "cambiar-a-verde")
Break 2 [4]> (transicion 'en-amarillo 'amarillo)

(EN-AMARILLO "accion-por-defecto")
Break 2 [4]> (transicion 'en-rojo 'verde)

(EN-ROJO "accion-por-defecto")
Break 2 [4]>
```

2do error de lógica:

```
(defun transicion (color-actual cambiar-a)
  (cond
    ((and (equal cambiar-a 'rojo) (equal color-actual 'en-amarillo)) (list color-actual "cambiar-a-rojo"))
    ((and (equal cambiar-a 'amarillo) (equal color-actual 'en-rojo)) (list color-actual "cambiar-a-amarillo"))
    ((and (equal cambiar-a 'verde) (equal color-actual 'en-amarillo)) (list color-actual "cambiar-a-verde"))
    (t (list color-actual "accion-por-defecto"))))
```

-El error consiste en la mala transición de los colores del semáforo, destacando su mala práctica en los ejemplos... Se solucionó, con las consultas al profesor.

Y luego de pasar a limpio las transiciones, quedaría de esta forma:

```
(defun transicion (color-actual cambiar-a)
  (cond
    ((and (equal cambiar-a 'verde) (equal color-actual 'en-rojo)) (list color-actual "cambiar-a-verde"))
    ((and (equal cambiar-a 'amarillo) (equal color-actual 'en-verde)) (list color-actual "cambiar-a-amarillo"))
    ((and (equal cambiar-a 'rojo) (equal color-actual 'amarillo)) (list color-actual "cambiar-a-rojo"))
    (t (list color-actual "accion-por-defecto"))))

rojo --> verde
verde --> amarillo
amarillo --> rojo
```

3er error de sintaxis:

```
(EN-VERDE "cambiar-a-amarillo")
Break 1 [3]> (transicion 'en-amarillo 'rojo)

*** - EVAL: la función EQUALL no está definida
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead of (FDEFINITION 'EQUALL).
RETRY          :R2      Reintentar
STORE-VALUE    :R3      Input a new value for (FDEFINITION 'EQUALL).
ABORT          :R4      Abort debug loop
Break 2 [4]> |
```

-Al probar los resultados de la función, luego de aplicar arreglos a los colores, me percate que en la última condicional ocurrió un error de sintaxis al escribir mal la función "equal", escribí "equall".

```
Break 1 [3]> (defun transicion (color-actual cambiar-a)
  (cond
    ((and (equal cambiar-a 'verde) (equal color-actual 'en-rojo)) (list color-actual "cambiar-a-verde"))
    ((and (equal cambiar-a 'amarillo) (equal color-actual 'en-verde)) (list color-actual "cambiar-a-amarillo"))
    ((and (equal cambiar-a 'rojo) (equall color-actual 'en-amarillo)) (list color-actual "cambiar-a-rojo"))
    (t (list color-actual "accion-por-defecto"))))
```

-Simplemente lo corregí, seguí con las pruebas y volvió a funcionar ahora correctamente.

Transición-Scheme:

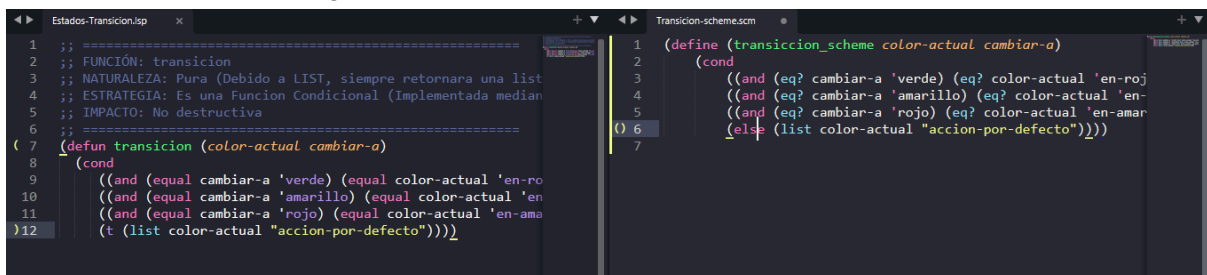
```
TRANSICION_SCHEME
- Prompt pasado a la IA Geminis para entender las funciones que se utilizaran:
Entiendo, ahora te voy a pasar, una lista de funciones y quiero que me expliques,
su función, su estructura y sintaxis en referencia al lenguaje scheme okay...
Lista de las funciones utilizadas en TRANSICION:
- Definición de Funciones: DEFUN
- Funciones Condicionales: COND
- Operadores Lógicos: AND, EQUAL
- Funciones Constructuras de listas: LIST
- Átomos, Strings, Listas
- T, NIL
Funciones de Scheme:
- (define (nombre-de-funcion parametro1 parametro2) (cuerpo-de-la-funcion)). Ejem:
- Scheme mantiene el cond de Lisp de manera casi idéntica para manejar múltiples ramas. Ejem:
- and: Funciona igual que en Lisp.
- equal?: En Scheme, por convención, las funciones que devuelven verdadero/falso terminan con un signo de interrogación ?.
- eq?: Compara si dos cosas son exactamente el mismo objeto en la memoria (muy rápido, usado para átomos o símbolos).
- Esta es la diferencia más grande e importante con Lisp. NIL significa lista vacía y falso al mismo tiempo, pero en Scheme...
Para Verdadero: Se usa #t (viene de true).
Para Falso: Se usa #f (viene de false).
Para la Lista Vacía: Se usa '() (un par de paréntesis evaluados literalmente).
-Componentes y Estructura en Scheme:
Todo esto se traslada directo desde Lisp. Usás list para construir una lista completa de un tirón.
también hace un uso intensivo del símbolo ' (quote o cita).
```

4to erro de sintaxis:

```
> (define (transicion_scheme color-actual cambiar-a)
  (cond
    ((and (eq cambiar-a 'verde) (eq color-actual 'en-rojo)) (list color-actual "cambiar-a-verde"))
    ((and (eq cambiar-a 'amarillo) (eq color-actual 'en-verde)) (list color-actual "cambiar-a-amarillo"))
    ((and (eq cambiar-a 'rojo) (eq color-actual 'en-amarillo)) (list color-actual "cambiar-a-rojo"))
    (else (list color-actual "accion-por-defecto"))))
> (transicion_scheme 'en-rojo 'verde)
*** ERROR IN transicion_scheme, console@3:10 -- Unbound variable: eq
1>
```

-Aquí sabía lo que necesitaba escribir según lo investigado, pero al ser lenguajes similares, me equivoque al escribir "eq" como en Lisp, pero en scheme es "eq?", ya que todos las funciones booleanas llevan un interrogante al final.

-El error se marca en el primero, pero yo los realice en todas las comparaciones, así que tuve que añadir la interrogante en todas:



```
1 ;; =====
2 ;; FUNCIÓN: transicion
3 ;; NATURALEZA: Pura (Debido a LIST, siempre retornara una list
4 ;; ESTRATEGIA: Es una Funcion Condicional (Implementada median
5 ;; IMPACTO: No destructiva
6 ;; =====
7 (defun transicion (color-actual cambiar-a)
8   (cond
9     ((and (equal cambiar-a 'verde) (equal color-actual 'en-ro
10      ((and (equal cambiar-a 'amarillo) (equal color-actual 'en
11      ((and (equal cambiar-a 'rojo) (equal color-actual 'en-ama
12      (t (list color-actual "accion-por-defecto")))))

1 (define (transicion_scheme color-actual cambiar-a)
2   (cond
3     ((and (eq? cambiar-a 'verde) (eq? color-actual 'en-roj
4     ((and (eq? cambiar-a 'amarillo) (eq? color-actual 'en-
5     ((and (eq? cambiar-a 'rojo) (eq? color-actual 'en-amar
6     (else (list color-actual "accion-por-defecto")))))
7
```

-Se guarda en una extensión .scm, que es la más normal de scheme, esta es una pequeña comparación entre ambas, pero el código scheme se ve así:

```
( 1 (define (transicion_scheme color-actual cambiar-a)
2   (cond
3     ((and (eq? cambiar-a 'verde) (eq? color-actual 'en-rojo)) (list color-actual "cambiar-a-verde"))
4     ((and (eq? cambiar-a 'amarillo) (eq? color-actual 'en-verde)) (list color-actual "cambiar-a-amarillo"))
5     ((and (eq? cambiar-a 'rojo) (eq? color-actual 'en-amarillo)) (list color-actual "cambiar-a-rojo"))
6     (else (list color-actual "accion-por-defecto"))))
7
```

-No está muy lejos de Lisp, ya que proviene del mismo, varias de sus formas estructurales son similares a la hora de hacer la función, algunos detalles se pueden notar mucho más, cuando definimos una función, también funciones predicados y funciones condicionales, cambian en ciertas cosas, poco perceptibles pero notables... Funcionamiento en práctica:

```
#<hilo #1 primordial>
> ( definir ( transicion_scheme color-actual cambiar-a )
  ( cond
    (( y ( eq? cambiar-a 'verde ) ( eq? color-actual 'en-rojo )) ( list color-actual
      "cambiar-a-verde" ))
    (( y ( eq? cambiar-a 'amarillo ) ( eq? color-actual 'en-verde )) ( list color-actual
      "cambiar-a-amarillo" ))
    (( y ( eq? cambiar-a 'rojo ) ( eq? color-actual 'en-amarillo )) ( list color-actual
      "cambiar-a-rojo" ))
    ( else ( lista color-actual "acción-por-defecto" ) )))
> ( transicion_scheme 'en-rojo 'verde )
(en-rojo "cambiar-a-verde" )
> ( transicion_scheme 'en-verde 'amarillo )
(en-verde "cambiar-a-amarillo" )
> ( transicion_scheme 'en-amarillo 'rojo )
(en-amarillo "cambiar-a-rojo" )
> ( transicion_scheme 'en-rojo 'amarillo )
(en-rojo "acción-por-defecto" )
>
```

Utilización del paquete local-time

-Es un biblioteca externa, desarrollada para fechas, horas, zonas horarias y duraciones de manera robusta, en mi caso, será utilizada para la muestra de fecha y hora, recibida de un formato unix epoch.

1er error de lógica:

```
cl-user> (registrar-cambio 1773274310 "verde" "amarillo")
2026-03-12T00:11:50.000000ZTiempo: [2026-03-12T00:11:50.000000Z], la luz ha cambiado de verde a amarillo
NIL
```

-El formato de salida, no era el especificado y salía dos veces en vez de una...

```
(defun registrar-cambio (epoch color-anterior color-nuevo)
  (format t "Tiempo: [~a], la luz ha cambiado de ~A a ~A~%"
    (local-time:format-timestring t (local-time:unix-to-timestamp epoch) :format local-time:+iso-8601-format+ )
    color-anterior color-nuevo))
```

-La solución a este problema fue cambiar el formato de salida de t, a nil tal que me permitía mandarlo como un string, pero aun así el formato de tiempo, no era el solicitado:

```
cl-user> (registrar-cambio 1773274310 "verde" "amarillo")
Tiempo: [2026-03-12T00:11:50.000000Z], la luz ha cambiado de verde a amarillo
NIL
```

2do erro de lógica:

-Como vemos, debemos solucionar el error de salida, para esto se debe de personalizar el mismo en el código...

```
cl-user> (registrar-cambio 1773274310 "verde" "amarillo")
Tiempo: [2026-03-12T00:11:50.000000Z], la luz ha cambiado de verde a amarillo
NIL
```

-Lo hacemos de la siguiente manera:

```
(defun registrar-cambio (epoch color-anterior color-nuevo)
  (format t "Tiempo: [~a], la luz ha cambiado de ~A a ~A~%"
    (local-time:format-timestring nil (local-time:unix-to-timestamp epoch)
      :format '(:year "-" :month "-" :day " " :hour ":" :min ":" :sec))
    color-anterior color-nuevo))
```

-En vez de poner los formatos que vienen por defecto en local-time, lo que hacemos es personalizar este mismo, simplemente agregando los cambios en medio de las variables, entre comillas. Dando como resultado:

```
cl-user> (registrar-cambio 1773274310 "verde" "amarillo")
Tiempo: [2026-3-12 0:11:50], la luz ha cambiado de verde a amarillo
NIL
```

Errores Extra:

-El siguiente error se genera al tratar de interpretar la función timer, dado que el símbolo o nombre timer ya está asociado a una función en particular de la librería local-time, esto también se debe a que el intérprete protege esos nombres ya asociados:

```
Lock on package SB-EXT violated when proclaiming TIMER as a function while
in
package COMMON-LISP-USER.
See also:
The SBCL Manual, Node "Package Locks"
```

```
(defun timer (timestamp)
  (cond
    ;; ciclo rojo: 87s + 3s
    ((< (mod timestamp 225) 87) 'rojo)
    ((< (mod timestamp 225) 90) 'rojo-intermitente)
    ;; ciclo verde: 117s + 3s
    ((< (mod timestamp 225) 207) 'verde)
    ((< (mod timestamp 225) 210) 'verde-intermitente)
    ;; ciclo amarillo: 3s + 3s por descarte
    ((< (mod timestamp 225) 213) 'amarillo)
    (t 'amarillo-intermitente)))
```

-Una solución a este problema, es desactivar temporalmente esa protección del intérprete para que la función timer, haga su correcto trabajo:

Con la sentencia **#+sbcl (sb-ext:unlock-package 'sb-ext)**, podemos desactivar temporalmente esta función.

```
#+sbcl (sb-ext:unlock-package 'sb-ext) ; Desactivamos el candado de SBCL para poder usar la palabra 'timer'
0 references | Inspect
(defun timer (timestamp)
  (cond
    ;; ciclo rojo: 87s + 3s
    ((< (mod timestamp 225) 87) 'rojo)
    ((< (mod timestamp 225) 90) 'rojo-intermitente)
    ;; ciclo verde: 117s + 3s
    ((< (mod timestamp 225) 207) 'verde)
    ((< (mod timestamp 225) 210) 'verde-intermitente)
    ;; ciclo amarillo: 3s + 3s por descarte
    ((< (mod timestamp 225) 213) 'amarillo)
    (t 'amarillo-intermitente)))
```

Bitácora - Cabrera Jorge

Bitácora:

Cabrera Jorge Sebastian/ Bitácora /Sistema De Auditoría:

Sistema de Auditoría

```
BSD-style licenses. See the CREDITS and COPYING files in the
distribution for more information.
* (defun registrar-cambio (epoch color-anterior color-nuevo)
  (format t
    "Tiempo ~A: la luz ha cambiado de ~A a ~A~%"
    epoch
    color-anterior
    color-nuevo))
REGISTRAR-CAMBIO
* (registrar-cambio 1000 'en-rojo 'en-verde)
Tiempo 1000: la luz ha cambiado de EN-ROJO a EN-VERDE
NIL
```

1er error de lógica:

```
* (defun registrar-cambio (epoch color-anterior color-nuevo)
  (format t
    "Tiempo ~A: la luz ha cambiado de ~A a ~A~%"
    epoch
    color-nuevo
    color-anterior))
REGISTRAR-CAMBIO
* (registrar-cambio 1000 'en-rojo 'en-verde)
Tiempo 1000: la luz ha cambiado de EN-VERDE a EN-ROJO
NIL
*
```

-El resultado no fue el esperado, ya que los colores aparecían invertidos en el registro de auditoría. Esto ocurría porque los parámetros color-anterior y color-nuevo fueron enviados en orden incorrecto a la función format.

```
* (defun registrar-cambio (epoch color-anterior color-nuevo)
  (format t
    "Tiempo ~A: la luz ha cambiado de ~A a ~A~%"
    epoch
    color-anterior
    color-nuevo))
REGISTRAR-CAMBIO
* (registrar-cambio 1000 'en-rojo 'en-verde)
Tiempo 1000: la luz ha cambiado de EN-ROJO a EN-VERDE
NIL
*
```

Se corrigió el orden de los parámetros, obteniendo el resultado esperado.

Error de Sintaxis:

```
* (defun registrar-cambio (epoch color-anterior color-nuevo)
  (format t
    "Tiempo ~A: la luz ha cambiado de ~A a ~A~%"
    epoch
    color-anterior
    color-nuevo))
REGISTRAR-CAMBIO
* (registrar-cambio 1000 en-rojo 'en-verde)

debugger invoked on a UNBOUND-VARIABLE @10006F5CA0 in thread
#<THREAD tid=5836 "main thread" RUNNING {1100C40003}>:
  The variable EN-ROJO is unbound.

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [CONTINUE   ] Retry using EN-ROJO.
  1: [USE-VALUE   ] Use specified value.
  2: [STORE-VALUE] Set specified value and use it.
  3: [ABORT       ] Exit debugger, returning to top level.

(SB-INT:SIMPLE-EVAL-IN-LEXENV EN-ROJO #<NULL-LEXENV>)
0] █
```

El programa no pudo ejecutarse correctamente debido a que el símbolo en-rojo fue escrito sin la comilla simple ('). Como consecuencia, Common Lisp intentó interpretarlo como una variable, pero dicha variable no estaba definida en el programa.

```
* (defun registrar-cambio (epoch color-anterior color-nuevo)
  (format t
    "Tiempo ~A: la luz ha cambiado de ~A a ~A~%"
    epoch
    color-anterior
    color-nuevo))
REGISTRAR-CAMBIO
* (registrar-cambio 1000 'en-rojo 'en-verde)
Tiempo 1000: la luz ha cambiado de EN-ROJO a EN-VERDE
NIL
* █
```

Se agregó la comilla simple faltante al símbolo en-rojo, permitiendo que la función se ejecutara correctamente y registrará el cambio de estado esperado.

Bitácora - Jeremias Fernandez Zalazar

Bitácora:

Jeremias Fernandez Zalazar / Bitácora/ Análisis de Ciclos semafóricos
(Duración de Ciclo, Recomendación de Ciclo):

1) Primera versión de la función **duración-ciclo:**

```
[1]> (defun duracion-ciclo (lista-segundos)
  (cond
    ((and (consp lista-segundos) (numberp (car lista-segundos) (cadr lista-segundos) (caddr lista-segundos)))
      (cond
        ((or (> (length lista-segundos) 3) (< (length lista-segundos) 3))
          (pprint "Error de parametro. La cantidad de numeros que representan a los segundos deben ser de 3
            (rojo->amarillo->verde)"))
          (t (+ (car lista-segundos) (cadr lista-segundos) (caddr lista-segundos))))
      )
    (t (pprint "Error, el parametro recibido no es una lista"))
  )
)
```

- Error de sintaxis:

```
[3]> (duracion-ciclo '(45 6 70))

*** - EVAL: se han entregado demasiados argumentos a NUMBERP:
```

- Con el objetivo de verificar que los elementos de la lista recibida sean números implemente la función numberp, pero no me di cuenta que la aplique a más de un argumento, olvidando aplicarla a cada uno de forma individual.

2) Segunda versión:

```
[1]> (defun duracion-ciclo (lista-segundos)
  (cond
    ((and (consp lista-segundos) (numberp (car lista-segundos)) (numberp (cadr lista-segundos)) (numberp (caddr lista-segundos))))
    (cond
      ((or (> (length lista-segundos) 3) (< (length lista-segundos) 3))
        (pprint "Error de parametro. La cantidad de numeros que representan a los segundos deben ser de 3
          (rojo->amarillo->verde)"))
        (t (+ (car lista-segundos) (cadr lista-segundos) (caddr lista-segundos))))
      )
    (t (pprint "Error, el parametro recibido no es una lista o alguno de sus elementos no es un numero"))
  )
)
```

- Error de sintaxis:

```
*** - SYSTEM:%EXPAND-FORM: (OR (> (LENGTH LISTA-SEGUNDOS) 3) (< (LENGTH LISTA-SEGUNDOS) 3)) debe ser una expresi lambda
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort debug loop
ABORT      :R2      Abort main loop
```

- Al corregir el error de sintaxis anterior, cometí uno más, dejando un paréntesis demás en el código, generando un error más grande.

3) Tercera versión:

```
[1]> (defun duracion-ciclo (lista-segundos)
  (cond
    ((and (consp lista-segundos) (numberp (car lista-segundos)) (numberp (cadr lista-segundos)) (numberp (caddr lista-segundos)))
      (cond
        ((or (> (length lista-segundos) 3) (< (length lista-segundos) 3))
          (pprint "Error de parametro. La cantidad de numeros que representan a los segundos deben ser de 3 (rojo->amarillo->verde)"))
        (t (+ (car lista-segundos) (cadr lista-segundos) (caddr lista-segundos))))
      )
    (t (pprint "Error, el parametro recibido no es una lista o alguno de sus elementos no es un numero")))
  )
)
```

- Error de sintaxis:

```
Break 2 [5]> (duracion-ciclo '(45 6 70))

*** - EVAL: la función NUMBER no está definida
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead of (FDEFINITION 'NUMBER).
RETRY          :R2      Reintentar
STORE-VALUE    :R3      Input a new value for (FDEFINITION 'NUMBER).
ABORT          :R4      Abort debug loop
ABORT          :R5      Abort debug loop
ABORT          :R6      Abort main loop
```

- Luego de corregir los paréntesis, por error puse un espacio entre “number” y “p” provocando que la función no sea reconocida por Lisp.

4) Cuarta versión:

```
[1]> (defun duracion-ciclo (lista-segundos)
  (cond
    ((and (consp lista-segundos) (numberp (car lista-segundos)) (numberp (cadr lista-segundos)) (numberp (caddr lista-segundos)))
      (cond
        ((or (> (length lista-segundos) 3) (< (length lista-segundos) 3))
          (pprint "Error de parametro. La cantidad de numeros que representan a los segundos deben ser de 3 (rojo->amarillo->verde)"))
        (t (+ (car lista-segundos) (cadr lista-segundos) (caddr lista-segundos))))
      )
    (t (pprint "Error, el parametro recibido no es una lista o alguno de sus elementos no es un numero")))
  )
)
```

- Error lógico:

```
Break 3 [6]> (duracion-ciclo '(45 6 70))

"Error de parametro. La cantidad de numeros que representan a los segundos deben ser de 3 (rojo->amarillo->verde)"
```

- Al olvidar poner “a que debe ser menor” la longitud de la lista recibida, la función no devuelve el resultado esperado, ya que siempre tomaba como “t” la sentencia (< (length lista-segundos)).

5) Quinta versión (versión final):

```
[1]> (defun duracion-ciclo (lista-segundos)
  (cond
    ((and (consp lista-segundos) (integerp(car lista-segundos)) (integerp(cadr lista-segundos)) (integerp(caddr lista-segundos)))
      (cond
        ((or (> (length lista-segundos) 3) (< (length lista-segundos) 3))
          (pprint "Error de parametro. La cantidad de numeros que representan a los segundos deben ser de 3
                    (rojo->amarillo->verde)"))
        (t (+ (car lista-segundos) (cadr lista-segundos) (caddr lista-segundos))))
      )
    (t (pprint "Error, el parametro recibido no es una lista o alguno de sus elementos no es un numero entero")))
  )
)
```

- Se corrigió el error anterior agregando un 3, para que la condición se evalúe correctamente. Además, cambié la función numberp por integerp, para asegurarme que no ingresen números que no fueran enteros. Logrando al fin la versión final y sin errores de “duración-ciclo”

```
Break 3 [6]> (duracion-ciclo '(45 6 70))
121
```

1) Primera versión de la función recomendación-ciclo(version final):

```
[1]> (defun recomendacion-ciclo (duracion-ciclo-segs)
  (cond
    ((integerp duracion-ciclo-segs)
      (cond
        ((< duracion-ciclo-segs 35) (pprint "La duracion del ciclo esta por debajo del recomendado,
          debe ser mayor a los 35 segundos"))
        ((> duracion-ciclo-segs 150) (pprint "La duracion del ciclo esta por encima del recomendado,
          debe ser menor a los 150 segundos"))
        (t (pprint "La duracion del ciclo se encuentra dentro de los estandares de ingenieria de trafico")))
      )
    (t (pprint "Error, el parametro recibido no es un numero")))
  )
)
```

- Esta función pude escribirla de forma correcta, logrando los resultados esperados sin problemas. La única modificación fue cambiar la función “numberp” por “integerp” para operar solamente con números enteros.

```
RECOMENDACION-CICLO
Break 3 [6]> (recomendacion-ciclo 120)
"La duracion del ciclo se encuentra dentro de los estandares de ingenieria de trafico"
Break 3 [6]> (recomendacion-ciclo 34)
"La duracion del ciclo esta por debajo del recomendado, debe ser mayor a los 35 segundos"
Break 3 [6]> (recomendacion-ciclo 151)
"La duracion del ciclo esta por encima del recomendado, debe ser menor a los 150 segundos"
```

Requerimiento 6- Jeremias Fernandez Zalazar

Bitácora:

Jeremias Fernandez Zalazar / Bitácora/ Informe de Distribucion Temporal:

1) Primera versión de la función **distribucion-temporal**:

```
[1]> (defun distribucion-temporal (regla-de-ciclo)
      (mapcar (lambda (regla-de-ciclo)
                (list (round (/ (* (reduce '+ regla-de-ciclo) regla-de-ciclo) 100)) "%"))
              )
      )
)
```

- Error de sintaxis:

```
[2]> (distribucion-temporal '(30 6 47))

*** - EVAL: no se han entregado suficientes argumentos a MAPCAR:
      (MAPCAR
        #'(LAMBDA (REGLA-DE-CICLO)
              (DECLARE
                (SYSTEM::SOURCE ((REGLA-DE-CICLO) (LIST (ROUND (/ (* (REDUCE '+ REGLA-DE-CICLO) REGLA-DE-CICLO) 100)) "%"))))
              (LIST (ROUND (/ (* (REDUCE '+ REGLA-DE-CICLO) REGLA-DE-CICLO) 100)) "%")))
      )
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort main loop
```

- En esta primera versión olvide incluir al final de la función “mapcar” la lista con la que debía operar; lo corregí simplemente escribiendo lo faltante.

2) Segunda versión:

```
Break 1 [3]> (defun distribucion-temporal (regla-de-ciclo)
      (mapcar (lambda (regla-de-ciclo)
                (list (round (/ (* (reduce '+ regla-de-ciclo) regla-de-ciclo) 100)) "%"))
              )
      regla-de-ciclo
      )
)
```

- Error lógico:

```
Break 1 [3]> (distribucion-temporal '(30 6 47))

*** - REDUCE: 30 is not a SEQUENCE
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort debug loop
ABORT      :R2      Abort main loop
```

- El error aquí fue poner la secuencia “reduce” dentro de “mapcar” y que ambas operen con la misma lista. Esto produce que “reduce” intente operar con un solo número en vez de con una secuencia. Esto lo resolví sacando la función “reduce” de dentro de “mapcar” y utilizando la función “let” para establecer una variable local y asignarle el valor deseado.

3) Tercera versión:

```
Break 4 [6]> (defun distribucion-temporal (regla-de-ciclo)
  (let ((total (reduce '+ regla-de-ciclo)))
    (mapcar (lambda (regla-de-ciclo)
              (list (round (/ (* regla-de-ciclo total) 100)) "%"))
            regla-de-ciclo)
  )
)
```

- Error lógico:

```
Break 4 [6]> (distribucion-temporal '(30 6 47))
((25 "%") (5 "%") (39 "%"))
```

- La función ya daba no daba error, pero la forma en la que pensé e implemente el cálculo del porcentaje era incorrecto, entregando un resultado alejado al esperado. Se corrigió de forma sencilla, cambiando de lugar la constante “100” y la variable “total”, aplicando de forma correcta la fórmula del porcentaje.

4) Cuarta versión:

```
Break 4 [6]> (defun distribucion-temporal (regla-de-ciclo)
  (let ((total (reduce '+ regla-de-ciclo)))
    (mapcar (lambda (regla-de-ciclo)
              (list (round (/ (* regla-de-ciclo 100) total)) "%"))
            regla-de-ciclo)
  )
)
```

- Ejecución:

```
Break 4 [6]> (distribucion-temporal '(30 6 47))
((36 "%") (7 "%") (57 "%"))
```

- “distribucion-temporal” ya funcionaba de forma correcta, entregando el resultado esperado; por lo que, a partir de esta versión solamente decidí mejorar la “robustez” del código, para que de un mensaje en pantalla en caso de recibir como parámetro algo que no cumpla con las condiciones.

5) Quinta versión (versión final):

```
[1]> (defun distribucion-temporal (regla-de-ciclo)
      (if (and (consp regla-de-ciclo) (= (length regla-de-ciclo) 3))
          (let ((total (reduce '+ regla-de-ciclo)))
              (list "Formato: ROJO --> AMARILLO --> VERDE"
                    (mapcar (lambda (regla-de-ciclo)
                              (list (float(/ (* regla-de-ciclo 100) total)) '%)
                                )
                            regla-de-ciclo)
                    )
          )
          (pprint "El parametro recibido no cumple con alguna condicion:
                  1) Ser una lista 2) Tener tres elementos")
          )
      )
)
```

- Ejecución:

```
[2]> (distribucion-temporal '(90 6 120))
("Formato: ROJO --> AMARILLO --> VERDE" ((41.666668 %) (2.777777 %) (55.555557 %)))

Break 8 [10]> (distribucion-temporal 30)
"El parametro recibido no cumple con alguna condicion:
  1) Ser una lista 2) Tener tres elementos"

Break 8 [10]> (distribucion-temporal '(2 3 4 5))
"El parametro recibido no cumple con alguna condicion:
  1) Ser una lista 2) Tener tres elementos"
```

- Como se ve en la imagen, agregue un “if” que verifique en el parámetro recibido cumpla con dos condiciones; 1) ser una lista 2) tener tres elementos. Además incluye un “list” con un string que indica a qué equivale cada porcentaje y deja saber al usuario el formato con el que se trabaja. También decidí cambiar la función “round” por “float” para asegurar que los porcentajes entregados por mi función sean lo más correcto posible.

Bitácora- Elorrieta Leandro Agustín

Errores en Lisp

```
Break 1 [3]> (defun timer (timestamp)
  (cond
    ((< (mod timestamp 216) 90) 'rojo)
    ((< (mod timestamp 216) 96) 'amarillo)
    (t 'verde)))

TIMER
Break 1 [3]> |
```

Error 1: Descalibración del ciclo total por omisión de la prórroga de seguridad (Iteración 2)

Causa del Error: Al diseñar la primera versión, calcule el módulo basándose únicamente en la suma de los tiempos fijos ($90 + 6 + 120 = 216$ segundos). Omití por completo el requerimiento de la Iteración 2, que exigía agregar 3 segundos de intermitencia de seguridad entre cada cambio de luz. Esto hacía que el ciclo se reiniciara antes de tiempo, impidiendo la correcta transición hacia los nuevos estados intermitentes.

Solución: Recalcule el ciclo completo sumando las tres intermitencias de 3 segundos cada una ($216 + 9 = 225$ segundos). Modifique el divisor de la primitiva matemática cambiando `mod timestamp 216` por `mod timestamp 225`.

```
Break 1 [3]> (defun timer (timestamp)
  (cond
    ((< (mod timestamp 225) 87) 'rojo)
    ((< (mod timestamp 225) 90) 'rojo-intermitente)
    ((< (mod timestamp 225) 93) 'amarillo)
    ((< (mod timestamp 225) 96) 'amarillo-intermitente)
    ((< (mod timestamp 225) 213) 'verde)
    (t 'verde-intermitente)))

TIMER
Break 1 [3]> (timer 92)

AMARILLO
```

Error 2: Secuencia incorrecta en el orden de prioridades de las luces urbanas

Causa del Error: Al estructurar las ramas condicionales del `cond`, arrastre un error de diseño conceptual ubicando la transición al color amarillo inmediatamente después del rojo. El sistema automatizado de tráfico urbano exige estrictamente la secuencia Rojo, Verde,

Amarillo. Al evaluar el amarillo antes, la logica del semaforo quedo completamente invertida y desincronizada con el flujo real de una interseccion vial.

Solucion: Reordene la cascada de evaluacion del cond. Coloque el filtro del tramo verde inmediatamente debajo del rojo (hasta el segundo 210) y delegue el intervalo final del ciclo para el color amarillo.

```
Break 1 [3]> (defun timer (timestamp)
  (cond
    (< (mod timestamp 225) 87) 'rojo)
    ((< (mod timestamp 225) 90) 'rojo-intermitente)
    ((< (mod timestamp 225) 207) 'verde)
    ((< (mod timestamp 225) 210) 'verde-intermitente)
    ((< (mod timestamp 225) 213) 'amarillo)
    (t 'amarillo-intermitente))

*** - SYSTEM::%EXPAND-FORM: (< (MOD TIMESTAMP 225) 90) debe ser una expresi lambda
Es posible continuar en los siguientes puntos:
```

Causa del Error: Al codificar el tramo correspondiente al color rojo, omití el paréntesis de apertura obligatorio. En la sintaxis de Lisp, cada cláusula del cond requiere ser una lista delimitada que vincule condición y retorno. Esta falta de integridad estructural provocó que el intérprete procesara la línea subsiguiente como parte del primer bloque, derivando en un fallo de expresión lambda.

Solución: Agregar el (al inicio de la primera condición para que quede con el doble paréntesis reglamentario: ((< (mod timestamp 225) 87) 'rojo)

```

Break 2 [4]> (defun timer (timestamp)
  (cond
    ((< (mod timestamp 225) 87) 'rojo)
    ((< (mod timestamp 225) 90) 'rojo-intermitente)
    ((< (mod timestamp 225) 207) 'verde)
    ((< (mod timestamp 225) 210) 'verde-intermitente)
    ((< (mod timestamp 225) 213 'amarillo))
    (t 'amarillo-intermitente)))

TIMER
Break 2 [4]> (timer 23)

ROJO
Break 2 [4]> (timer 212)

*** - <: AMARILLO is not a real number
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead.
ABORT          :R2      Abort debug loop
ABORT          :R3      Abort debug loop
Break 3 [5]> |

```

Causa: En la línea del amarillo, cerré el paréntesis del menor < después de la palabra 'amarillo en lugar de cerrarlo después del número 213. Al hacer 213 'amarillo), el intérprete asumió que el símbolo 'amarillo era parte de la comparación matemática, rompiendo la estructura de la cláusula.

Solución: Corregir el orden de los cierres para separar la condición del retorno, dejando 213) 'amarillo

```

Break 5 [7]> (defun timer (timestamp)
  (cond
    ((< (mod timestamp 225) 87) 'rojo)
    ((< (mod timestamp 225) 90) 'rojo-intermitente)
    ((< (mod timestamp 225) 207) 'verde)
    ((< (mod timestamp 225) 213) 'verde-intermitente)
    ((< (mod timestamp 225) 213) 'amarillo))
    (t 'amarillo-intermitente))

TIMER
Break 5 [7]> (timer 212)

*** - EVAL: la función T no está definida
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead of (FDEFINITION 'T).
RETRY          :R2      Reintentar
STORE-VALUE    :R3      Input a new value for (FDEFINITION 'T).
ABORT          :R4      Abort debug loop
ABORT          :R5      Abort debug loop
ABORT          :R6      Abort debug loop
ABORT          :R7      Abort debug loop
ABORT          :R8      Abort debug loop
Break 6 [8]> |

```

Explicación: Al probar el segundo 212, la función se rompió y entró en depuración (**Break 6**). **Esto demuestra el error:** como el cond se cerró antes en la línea de arriba por el paréntesis de más, el programa intentó ejecutar la letra T como si fuera una función, lo cual es inválido.

Errores en Scheme

```

> (define (timer timestamp)
  (cond
    ((< (modulo timestamp 225) 87) 'rojo)
    ((< (modulo timestamp 225) 90) 'rojo-intermitente)
    ((< (modulo timestamp 225) 207) 'verde)
    ((< (modulo timestamp 225) 210) 'verde-intermitente)
    ((< (modulo timestamp 225) 213) 'amarillo)
    else 'amarillo-intermitente)))
Exception: invalid syntax (cond ((< (...) 87) (quote rojo)) ((< (...) 90) (quote rojo-intermitente)) ((< (...) 207) (quote verde)) ((< (...) 210) (quote verde-intermitente)) ((< (...) 213) (quote amarillo)) ...)

```

Causa: En Scheme, la cláusula final **else** debe ir obligatoriamente entre paréntesis (**else ...**) para cerrar la estructura del condicional. Al dejarlo suelto, el compilador no puede parsear la sintaxis.

```

> (define (timer timestamp)
  (cond
    ((< (modulo timestamp 225) 87) ('rojo))
    ((< (modulo timestamp 225) 90) 'rojo-intermitente)
    (else 'amarillo-intermitente)))
> (timer 50)
Exception: attempt to apply non-procedure rojo
Type (debug) to enter the debugger.
>

```

Causa: Al envolver el símbolo 'rojo entre paréntesis extra ('rojo), **Scheme interpreta** que se quiere invocar a una función llamada "rojo". Como esa función no existe, el programa se interrumpe en tiempo de ejecución.

```

> (define (timer timestamp)
  (cond
    ((< (modulo timestamp 216) 87) 'rojo)
    ((< (modulo timestamp 216) 90) 'rojo-intermitente)
    ((< (modulo timestamp 216) 207) 'verde)
    ((< (modulo timestamp 216) 210) 'verde-intermitente)
    ((< (modulo timestamp 216) 213) 'amarillo)
    (else 'amarillo-intermitente)))
> (timer 218)
rojo
>

```

Causa: Al calcular el módulo, usé la sumatoria de los tiempos fijos originales (216 segundos) omitiendo sumar los 9 segundos de las nuevas prórrogas de seguridad de la Iteración 2. Esto hace que el ciclo se reinicie antes de tiempo, desfasando todo el comportamiento del semáforo.

```

> (define (timer timestamp)
  (cond
    ((< (modulo timestamp 225) 87) 'rojo)
    ((< (modulo timestamp 225) 90) 'rojo-intermitente)
    ((< (modulo timestamp 225) 120) 'verde)
    ((< (modulo timestamp 225) 210) 'verde-intermitente)
    ((< (modulo timestamp 225) 213) 'amarillo)
    (else 'amarillo-intermitente)))
> (timer 150)
verde-intermitente

```

Causa: Confundí la duración neta de la luz verde (120 segundos) con el límite acumulado en la línea de tiempo. Al poner < 120 de forma directa, el condicional empieza a contar desde el segundo 0 absoluto del ciclo, lo que recorta drásticamente el tiempo real que la luz verde fija debe permanecer encendida.

Bitácora - Lucia Valentina Garay

Bitácora:

Lucia Valentina, Garay - bitácora - ciclos-por-tiempo - informe

Error 1 (requerimiento 5) -reutilización de función

Error conceptual:

Inicialmente intente reutilizar la función duración-ciclos del requerimiento 4 para calcular la duración total del ciclo semafórico

```
[1]> (defun ciclos-por-tiempo (minutos)
  (truncate (/ (* minutos 60) (duracion-ciclos)))
)

WARNING: DEFUN/DEFMACRO: redefining function CICLOS-POR-TIEMPO in top-level, was defined in
C:\Users\Notebook\Desktop\Paradigmas yy Lenguaje\2026\TPI\Ciclos-por-tiempo
CICLOS-POR-TIEMPO
[2]> (defun duracion-ciclo (lista-segundos)
  (cond
    ((and (consp lista-segundos) (integerp (car lista-segundos)) (integerp (cadr lista-segundos)) (integerp (caddr lista-segundos)))
      (cond
        ((or (> (length lista-segundos) 3) (< (length lista-segundos) 3))
          (pprint "Error de parametro. La cantidad de numeros que representan a los segundos deben ser de 3 (rojo->amarillo->verde)"))
        (t (+ (car lista-segundos) (cadr lista-segundos) (caddr lista-segundos))))
      )
    (t (pprint "Error, el parametro recibido no es una lista o alguno de sus elementos no es un numero entero")))
  )
)

DURACION-CICLO
[3]> (ciclos-por-tiempo 15)

*** - EVAL: la función DURACION-CICLOS no está definida
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead of (FDEFINITION 'DURACION-CICLOS).
RETRY          :R2      Reintentar
STORE-VALUE    :R3      Input a new value for (FDEFINITION 'DURACION-CICLOS).
ABORT          :R4      Abort main loop
```

Problema:

La función requería una lista de entrada con los tiempos de cada color y eso complicaba innecesariamente el cálculo

Solución:

Se decidió utilizar directamente el valor fijo de 216 segundo definido por la consiga, haciendo la función más simple e independiente

```
Break 3 [6]> (defun ciclos-por-tiempo (minutos)
  (truncate (/ (* minutos 60) 216))
)

CICLOS-POR-TIEMPO
Break 3 [6]> (ciclos-por-tiempo 15)

4 ;
1/6
```

Error 2 -Validación de datos

Error lógico:

La primera versión no verifica si el parámetro recibido era un número

```
Break 4 [7]> (defun ciclos-por-tiempo (minutos)
  (truncate (/ (* minutos 60) 216))
)

CICLOS-POR-TIEMPO
Break 4 [7]> (ciclos-por-tiempo 'hola)

*** - *: HOLA is not a number
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead.
ABORT          :R2      Abort debug loop
ABORT          :R3      Abort debug loop
ABORT          :R4      Abort debug loop
ABORT          :R5      Abort debug loop
ABORT          :R6      Abort main loop
```

Problema:

Provocaba errores durante el cálculo

Solución:

Se agrego la funcion “numberp” para validar que el dato ingresado sea numérico antes de realizar las operaciones

```
Break 5 [8]> (defun ciclos-por-tiempo (minutos)
  (if (numberp minutos)
      (truncate (/ (* minutos 60) 216))
      "ingrese un dato valido"
  )
)

CICLOS-POR-TIEMPO
Break 5 [8]> (ciclos-por-tiempo 15)

4 ;
1/6
Break 5 [8]> (ciclos-por-tiempo 'hola)

"ingrese un dato valido"
Break 5 [8]> |
```

Error 3 - bitácora- Informe - uso de local-time

Error conceptual:

Inicialmente no comprendía el uso del parámetro nil dentro del local-time:format-timestring.
(local-time:format-timestring nil ...)

<https://local-time.common-lisp.dev/manual.html>

Solución:

Se investigó la documentación de la librería y se verificó que nil permite devolver la fecha y hora como una cadena de texto para luego escribirla en el archivo mediante format

Error 4 - Librería local - time no encontrada

Problema

Al intentar ejecutar la función “informe”, el intérprete devolvía;

```
Break 1 [3]> (defun informe (datos)
  (with-open-file
    (stream "informe-ejecucion-semaforo.txt"
      :direction :output ;abre el archivo para escribir
      :if-exists :supersede) ;si ya existe, se reemplaza por completo

    (format stream "Informe de Ejecucion del Sistema Semaforico~%" ) ; escribe una linea en el archivo
    (format stream "=====~%")

    (mapcar
      (lambda (x)
        (format stream "~a - Transicion ~A --> ~A~%"
          (local-time:format-timestring
            nil ;para que devuelva la fecha y hora como una cadena de texto, con el local-time:format-timestring
            (local-time:unix-to-timestamp (car x)) ;convierte el tiempo Unix a el formato fecha y hora
            :format '(:year "-" :month "-" :day
              " "
              :hour ":" :min ":" :sec))
            (cadr x) ;color anterior
            (caddr x))) ;color nuevo
          datos)

        (format stream "~%--- Fin del Informe ---")))))

*** - READ en
#<INPUT CONCATENATED-STREAM
#<IO TWO-WAY-STREAM #<INPUT UNBUFFERED FILE-STREAM CHARACTER @29> #<OUTPUT UNBUFFERED FILE-STREAM CHARACTER>>>
: no existe ningun paquete con el nombre "LOCAL-TIME"
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort debug loop
```

no existe ningún paquete con el nombre “LOCAL - TIME”

Causa:

La función utiliza la librería externa local-time, pero dicha librería no estaba instalada de forma local

Solución:

Se verificó que el código era correcto y que el problema solo pertenecía al entorno de ejecución. Para utilizar la función correctamente es necesario contar con Quicklisp y la librería “local-time” instalados

Conclusion Grupal

CONCLUSION - GRUPO 21

Este trabajo permitió aplicar los conceptos vistos en la materia mediante el desarrollo de un sistema semafórico en Common Lisp.

Durante el proceso se implementaron los requerimientos solicitados, se realizaron modificaciones en distintas iteraciones y se resolvieron errores surgidos durante el desarrollo.

La realización del proyecto ayudó a reforzar el uso de funciones, estructuras condicionales, manejo de listas y otros conceptos propios de la programación funcional

Britez, Lautaro Tomas

Cabrera, Jorge Sebastian

Elorrieta, Leandro Agustin

Fernandez, Zalazar Jeremias

Garay, Lucia Valentina

Bibliografía

Bibliografía:

- **Quicklisp.** (s.f.). *Quicklisp beta release*. <https://www.quicklisp.org/beta/>
- **Scheme.org.** (s.f.). *Scheme: An elegant and mature dialect of Lisp*. <https://www.scheme.org/>
- **Sharp Lispers.** (s.f.). local-time: A date and time library for Common Lisp [Repositorio de GitHub]. GitHub. <https://github.com/sharplispers/local-time.git>
- **Borodkin, S., & Sharp Lispers.** (s.f.). Local-time Manual. Common Lisp. <https://local-time.common-lisp.dev/manual.html>